

Lab 03: State Estimation and Belief Updating

Objective

Implement and visualize belief updating using the A-matrix likelihood, compare inference with different observation noise levels, and analyze convergence behavior.

Prerequisites

- Completed Labs 01–02
- Understanding of Bayesian inference and VFE
- Access to the `active_inference` library

Part 1: Basic State Inference

Goal: Run state inference on a simple 2-state system and examine the posterior.

1. Create a `GenerativeModel` with a clear A-matrix ($A = \begin{bmatrix} 0.9 & 0.1 \\ 0.1 & 0.9 \end{bmatrix}$).
2. Run `run_state_inference()` with a uniform prior and observation $z_0 = 0$.
3. Print the posterior, number of iterations, and whether convergence was achieved.

```
import numpy as np
from active_inference.agent import GenerativeModel
from active_inference.math import run_state_inference

# TODO: Create model and run state inference
# TODO: Print result["q_s"], result["converged"], result["num_iters"]
```

Response:

Write your response here

Part 2: Sequential Observation Updates

Goal: Update beliefs across multiple observations.

1. Start with a uniform prior $q(s) = [0.5, 0.5]$.
2. Process the observation sequence $[0, 0, 1, 0, 1, 1]$.
3. After each observation, use the posterior as the new prior.
4. Record and plot the belief trajectory using `plot_beliefs()`.

```
from active_inference.visualization import plot_beliefs

beliefs_history = []
prior = model.D.copy()
for obs in [0, 0, 1, 0, 1, 1]:
    result = run_state_inference(prior=prior, observation=obs, A=model.A)
    prior = result["q_s"]
    beliefs_history.append(prior.copy())

plot_beliefs(beliefs_history, state_labels=["s0", "s1"],
             save_path="output/lab03_beliefs.png")
```

Response:

Write your response here

Part 3: Noise Comparison

Goal: Compare inference under different A-matrix noise levels.

1. Define three A-matrices: clear (\$0.95/0.05\$), moderate (\$0.75/0.25\$), noisy (\$0.55/0.45\$).
2. For each, run inference on the same observation sequence $[0, 0, 0, 0, 0]$.
3. Plot the posterior $q(s_0)$ after each observation for all three noise levels on the same graph.

Response:

Write your response here

Part 4: Convergence Analysis

Goal: Visualize and analyze inference convergence.

1. Run `run_state_inference()` with a tight threshold (10^{-12}) and 50 max iterations.
2. Plot the convergence curve using `plot_convergence()`.
3. Repeat with the noisy A-matrix and compare convergence speed.

```
from active_inference.visualization import plot_convergence

result = run_state_inference(
    prior=model.D.copy(), observation=0, A=model.A,
    num_iterations=50, convergence_threshold=1e-12,
)
plot_convergence(result["delta_history"], threshold=1e-12,
                 save_path="output/lab03_convergence.png")
```

Response:

Write your response here

Part 5: Prediction Errors in a Live Agent

Goal: Run an agent and examine prediction errors.

1. Create an `ActiveInferenceAgent` and a `DiscreteEnvironment`.
2. Run 20 steps of the perception-action loop.
3. At each step, compute `agent.prediction_error(obs)` and store it.
4. Visualize with `plot_prediction_errors()`.

```
from active_inference.visualization import plot_prediction_errors

# TODO: Run loop, collect observations and predicted observations
# TODO: Call plot_prediction_errors()
```

Response:

Write your response here

Summary

Skill	Library Component	Status
Run standalone state inference	<code>run_state_inference()</code>	
Update beliefs across sequential observations	Prior chaining	
Compare inference under different noise levels	Varying A-matrix	
Visualize convergence diagnostics	<code>plot_convergence()</code>	
Compute and plot prediction errors	<code>agent.prediction_error()</code> , <code>plot_prediction_errors()</code>	